

Project Part 2: Detecting Python Security Smells using CodeQL

This project explores the use of CodeQL for static analysis of Python programs to detect security smells in publicly available GitHub gists. Security Smells are patterns in code that indicate potential security vulnerabilities or insecure practices that may lead to cyberattacks. The dataset is provided, and I will give a table with each detected smell, a description and the count. Custom CodeQL queries were developed to detect multiple types of security smells, including command injection, dynamic code execution, insecure HTTP usage, and many more. The queries were executed on the dataset and results were collected including file paths, line numbers, and matched code snippets. The findings were manually reviewed to confirm if they were actual security risks.

A table summarizing the detected smells, descriptions, and counts is included below.

Security Smell	Description	Count
Exec usage	Use of exec to execute dynamically generated code which can lead to arbitrary code execution	33
Eval Usage	Use of eval to evaluate user input	127
Command Injection	Use of os.system or subprocess with unsanitized input	34
HTTP without TLS	Use of http:// instead of https://	2561
Bad File permission	Use of chmod with overly permissive settings	1
Ignoring Exception Handling	Use of try/except blocks that may ignore errors	5485
Hardcoded Temp Directory	Use of fixed paths like /tmp/, which can lead to file manipulation	32
Hardcoded Secrets	Sensitive data such as API keys or credentials in code	2584
Empty Password	Password variables initialized as empty strings	581
No Certificate Validation	Disabling SSL verification allows MITM attacks	0

Manual Validation

The detected results from each query were manually inspected to verify whether they represent actual security smells. The validation process involved reviewing the matched code snippets and analyzing the context to see if they contained vulnerabilities. From the found smells I found that most of them did contain valid security smells, while a small number were false positives but still indicated potentially insecure coding practices.